Integrantes: Heidi Villavicencio | Steeven Toala | Nixon Hurtado | Andrea Vásquez

1. TÍTULO

Denominación Oficial del Entregable: Integración del Editor TipTap con Autoguardado Debounce – Código EDT: 3.6

2. INTRODUCCIÓN

Las secciones descriptivas de los formatos SGC institucionales (e.g., Justificación Académica, Metodología de Cátedra, Medios de Evaluación) demandan formateo de texto estructurado (negritas, cursivas, listas ordenadas y tablas) manteniendo completa compatibilidad semántica con los estándares tipográficos oficiales de la UTA. Para evitar la pérdida de información ante cierres imprevistos de navegador, se integró el editor moderno basado en ProseMirror denominado TipTap en React, complementado con una estrategia de autoguardado asíncrono con temporizador debounce (800 ms) y sanitización estricta contra inyección XSS mediante DOMPurify.

3. OBJETIVO

3.1. Objetivo General

Integrar y configurar el editor de texto enriquecido TipTap con mecanismo de autoguardado debounce de 800 ms y vista previa A4 oficial en el sistema SIGE.

3.2. Objetivos Específicos

- Implementar el componente de edición enriquecida con extensiones oficiales de TipTap.
- Desarrollar el hook de autoguardado reactivo con debounce de 800 ms y feedback de estado.
- Integrar sanitización de contenido HTML y renderizado en vista preliminar A4.

4. ALCANCE

4.1. Límites del Entregable (In-Scope)

- Formateo tipográfico (párrafos, encabezados H1-H3, listas, citas, negritas).
- Indicador visual dinámico de guardado (*"Guardando cambios..."* → *"Guardado en tiempo real"*).
- Vista previa fidedigna en formato A4 con membrete institucional UTA.

4.2. Exclusiones (Out-of-Scope)

- Edición colaborativa simultánea en tiempo real vía WebSockets (reservada para Fase 5).

5. DEFINICIONES

- **TipTap / ProseMirror:** Motor de edición de texto enriquecido headless, extensible y fuertemente tipado para aplicaciones web modernas.
- **Debounce (Técnica):** Patrón de programación que retrasa la ejecución de una función hasta que haya transcurrido un intervalo de inactividad predefinido (800 ms).
- **DOMPurify:** Librería de sanitización de código HTML que elimina vectores de ataque Cross-Site Scripting (XSS).

6. RESPONSABILIDADES

- **Andrea Vásquez (Responsable EDT 3.6):** Integración del editor TipTap, extensiones y maquetación de la vista A4.
- **Steeven Toala (Líder Técnico):** Implementación del hook de debounce y sanitización con DOMPurify.
- **Ing. Paulo Torres (Docente Evaluador):** Aprobación de la fidelidad del texto frente a los formatos SGC.

7. NORMATIVA LEGAL Y TÉCNICA

7.1. Marco Legal Ecuatoriano

- **Ley Orgánica de Educación Superior (LOES), Art. 122:** Conservación fidedigna de contenidos de planificación académica.
- **Ley Orgánica de Protección de Datos Personales (LOPDP), Art. 32.**

7.2. Normas Técnicas Internacionales

- **W3C HTML5 Specification:** Semántica y estructuración de contenido web limpio.
- **OWASP Top 10: A03 – Injection:** Sanitización estricta de entradas de texto enriquecido.

8. MÉTODO, POLÍTICAS Y CONTENIDOS

8.1. Enfoque Metodológico

Se configuró TipTap como editor *headless* desacoplado con estilos Tailwind Typography (`prose prose-sm`). Cada pulsación de tecla desencadena la actualización del árbol DOM local; el

hook `useDebounce` reinicia un temporizador de 800 ms. Al vencer el temporizador, el contenido HTML es sanitizado con DOMPurify y transmitido vía HTTP PUT al endpoint `/api/secciones/:id`. En caso de fallo de red, el contenido se respalda de forma transparente en `localStorage` para recuperación offline inmediata.

8.2. Políticas de Calidad de la Edición de Texto

1. **Política de Autoguardado Silencioso:** La persistencia debe ocurrir en segundo plano sin bloquear la escritura ni provocar parpadeos visuales en pantalla.
2. **Política de Sanitización Mandatoria:** Ningún fragmento HTML se almacena en base de datos sin pasar previamente por el filtro anti-XSS de DOMPurify.
3. **Política de Respaldo Local (RPO = 0):** Si el servidor no responde, el borrador se custodia localmente hasta que se restablezca la conectividad.
4. **Política de Formato Limpio:** El editor prohíbe etiquetas complejas ajenas al estándar institucional, garantizando exportaciones PDF libres de distorsión tipográfica.

9. DESCRIPCIÓN DE ACTIVIDADES

La redacción de las secciones descriptivas de los planes docentes (justificación, metodología y compromisos académicos) requirió un editor que garantizara tipografía homogénea y persistencia asíncrona segura. Las actividades ejecutadas comprendieron:

- **Integración del Núcleo TipTap sobre la Aplicación React:** Se incorporó la suite modular `@tiptap/react` con extensiones oficiales esenciales (`StarterKit`, `Table`, `Link`), configurando una barra de herramientas enriquecida con formatos autorizados por el SGC.
- **Serialización Estructurada en Formato JSONB:** En lugar de almacenar cadenas HTML planas susceptibles a inyecciones de código y difíciles de paginar en reportes PDF, el editor serializa el contenido en un árbol de nodos JSON nativo, persistido en la columna `contenido_jsonb` de PostgreSQL 18.
- **Mecanismo de Autoguardado Asíncrono con Debounce de 1500 ms:** Mediante el hook `useDebounce`, el sistema reinicia el contador mientras el usuario escribe activamente; al detectar 1500 ms de inactividad tras la última pulsación, despacha la petición de guardado al backend sin interrumpir la experiencia de redacción.
- **Indicador Visual Reactivo del Estado de Persistencia:** Se implementó un componente de estado en la cabecera del editor que informa de forma no intrusiva si el borrador se está guardando, si la persistencia fue exitosa o si existe una anomalía de red.

- **Sanitización Perimetral de Contenido con DOMPurify:** Como medida de protección contra ataques XSS, el contenido se procesa a través de un pipeline de sanitización estricta que remueve cualquier etiqueta potencialmente peligrosa antes del guardado en el servidor.

9.1. Matriz Técnica de Configuración del Editor de Texto Enriquecido

A continuación se detalla la matriz de especificación técnica del editor TipTap y su mecanismo de autoguardado:

Módulo Editor	Librería Base	Funcionalidad SGC Brindada	Seguridad
Editor Básico	TipTap StarterKit	Párrafos, negrita, cursiva, listas ordenadas y viñetas.	Estándar W3C
Encabezados	TipTap Heading	Subsecciones estructuradas (niveles H2 y H3).	Filtrado W3C
Tablas Dinámicas	TipTap Table	Tablas con celdas combinadas para anexos descriptivos.	Limpieza scripts
Texto Guía	TipTap Placeholder	Texto guía institucional cuando la sección está vacía.	No persistido
Control Longitud	TipTap CharCount	Límite máximo de 2500 palabras por sección institucional.	Validación cliente

Tabla 1: Extensiones habilitadas en el editor enriquecido TipTap.

9.2. Matriz de Transición del Indicador de Autoguardado

Estado	Texto del Indicador	Icono / Color	Condición Técnica de Disparo
Inactivo	***"Todo guardado"***	Check verde	El contenido en pantalla coincide exactamente con la base de datos.
Escritura	***"Guardando cambios..."***	Spinner azul	El usuario tipea; temporizador debounce activo (< 800 ms).
Éxito	***"Guardado en tiempo real"***	Nube verde	Petición PUT confirmada con código HTTP 200 OK.
Error	***"Sin conexión - Guardado local"***	Alerta ámbar	Error de red; respaldo asegurado en `localStorage`.

Tabla 2: Estados visuales del mecanismo de autoguardado.

10. FIRMAS DE RESPONSABILIDAD

ACCIÓN	NOMBRE Y CARGO	FIRMA	FECHA
Elaborado por:	Heidi Villavicencio Analista de Calidad / UI-UX		21/09/2026
Revisado por:	Steeven Toala Líder de Proyecto / Arquitecto		21/09/2026
Aprobado por:	Ing. Paulo Torres, Mg. Docente Evaluador – Gestión Proyectos	[PENDIENTE DE APROBACIÓN]	**Pendiente**

11. CONTROL DE CAMBIOS

Versión	Descripción del cambio	Responsable del cambio	Fecha de aprobación
1.0	Elaboración y entrega del artefacto técnico inicial	Steeven Toala / Heidi Villavicencio	21/09/2026

12. ANEXOS

Anexo 1: Desglose Metodológico y Procedimiento Técnico de Construcción

La redacción de las secciones descriptivas (justificación, metodología y compromisos académicos) requirió un editor que garantizara tipografía estandarizada y persistencia sin riesgo de pérdida de datos:

1. **Integración del Núcleo TipTap sobre React:** Se incorporó `@tiptap/react` con extensiones oficiales esenciales: `StarterKit` (títulos de nivel 2 y 3, negrita, cursiva, listas viñeteadas y ordenadas), `Table` y `Link` con apertura segura (`rel="noopener noreferrer"`).
2. **Estructura de Datos Semi-estructurada JSONB:** En lugar de almacenar cadenas HTML arbitrarias propensas a ataques XSS y difíciles de renderizar en PDF vectorial, el editor serializa el documento en un árbol de nodos JSON nativo (*TipTap JSON Schema*), persistido en la columna `contenido_jsonb` de PostgreSQL 18.
3. **Autoguardado con Debounce de 1500 ms:** Se implementó el hook `useDebounce`. Mientras el docente escribe, los temporizadores se reinician; tras 1500 ms de inactividad, se despacha

la petición asíncrona al backend.

4. **Indicador Visual de Estado de Persistencia:** En la esquina superior derecha del editor se presenta un badge dinámico con tres estados:
   - **Icono Azul Giratorio:** ”Guardando cambios en servidor...”.
   - **Icono Verde con Check:** ”Borrador guardado automáticamente (hace X seg)”.
   - **Icono Rojo de Alerta:** .Error de conexión: reintentando almacenamiento local”.
5. **Sanitización Perimetral con DOMPurify:** Como doble barrera contra ataques de inyección, el contenido es sanitizado antes del renderizado en cliente y antes de su almacenamiento en base de datos.

Anexo 2: Evidencia Visual de Funcionamiento Operativo



Figura 1: Editor TipTap de Texto Enriquecido con Maquetación A4 y Autoguardado Activo.

Desglose Técnico y Análisis Estructural del Artefacto Visual

- **Elementos Estructurales Visualizados:** La imagen ilustra la edición del apartado descriptivo del Plan de Trabajo Docente. Se observa la barra de herramientas del editor TipTap (encabezados, negrita, cursiva, viñetas, tablas), el lienzo con proporciones oficiales de hoja A4 y el indicador de autoguardado activo.
- **Mecanismo de Autoguardado Asíncrono con Debounce:** Conforme el docente redacta, el contenido se serializa en formato JSONB estructurado. Un hook de debounce retarda la petición de red durante 1500 ms tras la última pulsación de tecla, enviando la actualización sin saturar el servidor ni interrumpir la escritura.

- **Fidelidad Visual y Erradicación de Inconsistencias Word:** La previsualización A4 en tiempo real garantiza que el formato visual del texto editado coincida con exactitud milimétrica con el documento final exportado en PDF, eliminando desconfiguraciones tipográficas.
- **Trazabilidad con el Código Fuente:** Implementado en `frontend/src/components/editor/TipTapEditor.jsx` y el hook `useAutosave.js`.

Anexo 3: Matriz de Verificación Operacional del Editor y Autoguardado

Escenario de Prueba	Condición de Entrada y Acción	Comportamiento Verificado en Editor	Resultado Verificado
P1: Ráfaga de Pulsaciones	Escritura sostenida de 250 palabras sin pausas.	0 peticiones intermedias; se dispara exactamente una sola llamada tras 1500 ms de inactividad.	Tráfico Optimizado
P2: Inyección de Script XSS	Pegado de texto con etiqueta oculta `<script>alert(1)</script>`.	Etiqueta neutralizada y eliminada por sanitizador; texto plano conservado.	Sanitización Exitosa
P3: Simulación Offline	Desconexión de red durante la redacción de un párrafo.	Notificación en amarillo; almacenamiento en cola local y envío automático al reconectar.	Resiliencia Offline
P4: Formato Enriquecido	Aplicación de negritas, listas ordenadas y tablas.	Estructura JSONB preservada fielmente al recargar el expediente desde el motor.	Formato Consistente

Tabla 3: Matriz de pruebas del editor de texto enriquecido y mecanismo de autoguardado.